The 262-operation $\langle 4 \times 4 \times 4 : 48 \rangle$ algorithm is also the more accurate one: error-bound exponent 9/4 and about a quarter less error

Greg Sidebottom

Draft — October 25, 2026

Abstract

Dumas, Pernet and Sedoglavic's accurate rational $\langle 4 \times 4 \times 4 : 48 \rangle$ algorithm (Table 1 of their 2026 paper; `4x4x4_48_accurate` in PLinOpt) has $(\infty, 2)$-growth factor $\gamma_{\infty,2} = 18\sqrt{2} \approx 25.46$, an error-bound exponent $\log_4 \gamma_{\infty,2} \approx 2.335$. Their later 262-operation variant `4x4x4_48_accurate_262`, in the same orbit, was published for its lower operation count. We show that it is also the more accurate algorithm. Its growth factors are smaller in all four norms they consider: $\gamma_{\infty,2} = 16\sqrt{2}$, an exponent of exactly 9/4; $\gamma_{\infty,\infty} = 132$ against 148; $\gamma_{2,\infty} = 16\sqrt{963}$ against $8\sqrt{4142}$; and $\gamma_{2,2} = 64\sqrt{2}$ against $8\sqrt{130}$. Its $\gamma_2 = 64(1+\sqrt{2})$ is the same, and its rounding-chain factor Q_0 is 80 against 180. Measured against exact references on random matrices, in single and double precision at one to three recursion levels, its error is 0.75–0.77 of `4x4x4_48_accurate`'s on average (0.62–0.90 for single instances), at the same speed in a fused-pass implementation over Apple's Accelerate. In recursion down to scalars its advantage grows, to 0.72 at five levels. We found a scheme equivalent to `4x4x4_48_accurate_262` independently, by minimizing $\gamma_{\infty,2}$ over the de Groote orbit and the scheme's three role assignments with an exact-gradient search: in the published role the lowest point the search finds is equivalent to it, and its coefficients are dyadic. Among the orbit's 648 dyadic orthogonal points, enumerated exhaustively, the points that minimize both $\gamma_{\infty,2}$ and $\gamma_{\infty,\infty}$ are all equivalent to it. The orbit's global $\gamma_{\infty,2}$ minimum, 21.41, needs another role assignment and coefficients in $\mathbb{Q}(\sqrt{2})$, and its measured error is no lower than `4x4x4_48_accurate`'s. Dyadic probe points show that neither $\gamma_{\infty,2}$ nor $\gamma_{\infty,\infty}$ alone predicts the measured error, while Q_0 and exact dyadic coefficients matter. The other known rational classes, Li, Wang and Hu's curve and Agarwal's scheme, have larger $\gamma_{\infty,2}$ minima.

1 Introduction

Two 4×4 matrices can be multiplied with 48 multiplications over any ring containing 1/2. AlphaEvolve found a rank-48 scheme with complex coefficients [14], and Dumas, Pernet and Sedoglavic (DPS) turned it into one with rational coefficients [7]. Applied recursively, such a scheme multiplies $n \times n$ matrices with $O(n^{\log_4 48}) = O(n^{2.7925})$ operations. Fast schemes trade accuracy for speed, and their error bounds are governed by a *growth factor* computed from the scheme's coefficient matrices [3, 2, 5, 12]. DPS generalized it to a (p, q)-growth factor $\gamma_{p,q}$ for any output and input norms [6, 9], and they argue that the $(\infty, 2)$-factor $\gamma_{\infty,2}$ best reflects the accuracy observed in practice.

In [8] DPS proposed an accurate variant of their rational scheme, given as Table 1 of that paper and in PLinOpt as `4x4x4_48_accurate` [11]. They found it in the de Groote isotropy orbit [4] of their first scheme: a point minimizing the smooth bound $\gamma_2 \geq \gamma_{2,2}$ along the orbit, then moved by orthogonal isotropies, which keep γ_2, to get sparse straight-line programs. Its growth factor is

$\gamma_{\infty,2} \approx 25.456$, error-bound exponent $\log_4 \gamma_{\infty,2} \approx 2.335$, the smallest among the algorithms they compare, and its programs take 284 operations. The paper's versions run from 19 March 2026 (v1) to 29 July 2026 (v3, the current one), and `4x4x4_48_accurate` entered PLinOpt on 23 June 2026.

A cheaper accurate variant, with 262 operations, came next. Its earliest trace is the second version of [10], dated 2 October 2026, whose introduction states, without a reference or coefficients, that the best currently known accurate variant has leading term 9+3/16, or 5+3/4 in an alternative basis. The leading term is $1 + q/32$ for q additions, subtractions and scalings, so $9 + 3/16$ means $q = 262$. The first version, of 10 September 2026, mentions only `4x4x4_48_accurate` (leading term 9.875). The scheme itself was first published in PLinOpt on 13 October 2026 as `4x4x4_48_accurate_262` (commit `b441528`, by J.-G. Dumas), described as "accurate with 262 operations, or (5+3/4) ALT/CoB". Neither source says how it was found or reports its growth factors, and `4x4x4_48_accurate` remains the accurate variant on record.

We call schemes by their PLinOpt names: `4x4x4_48_accurate` for Table 1 of [8], `4x4x4_48_accurate_262` for the variant with 262 operations, `4x4x4_48_rational` for the first rational scheme [7], `4x4x4_48_201` for our 201-operation point [16], and `2x2x2_7_Winograd`, `2x2x2_7_Strassen` and `2x2x2_7_DPS-accurate` for the $\langle 2 \times 2 \times 2 : 7 \rangle$ algorithms. Tables drop the common prefixes. Points of our search that are not in PLinOpt have descriptive names.

We show that `4x4x4_48_accurate_262`, the cheaper variant, is also the more accurate one:

- Its growth factors are smaller than `4x4x4_48_accurate`'s in all four norms of [8, Table 2], with the same γ_2: $\gamma_{\infty,2} = 16\sqrt{2}$, an error-bound exponent of exactly 9/4, against 25.456; $\gamma_{\infty,\infty} = 132$ against 148; $\gamma_{2,2} = 64\sqrt{2}$ against 91.21; and $\gamma_{2,\infty} \approx 496.5$ against 514.9. Its rounding-chain factor Q_0 is 80 against 180 (Section 3).

- Measured on random matrices against exact references, in single and double precision, its error is 0.75–0.77 of `4x4x4_48_accurate`'s on average at one to three recursion levels, and never above 0.90. In recursion down to scalars the advantage grows, to 0.72 at five levels. It runs at the same speed in a fused-pass implementation over Apple's Accelerate (Sections 5 and 6).

Before we knew of `4x4x4_48_accurate_262`, which was first published in PLinOpt (commit `b441528` [11]), our search found a scheme equivalent to it, that is, `4x4x4_48_accurate_262` written in another way (Section 2.3). The search minimizes $\gamma_{\infty,2}$ over the orbit and the scheme's role assignments (Section 4). That search is the paper's other contribution. In the published role the lowest point the search finds is equivalent to `4x4x4_48_accurate_262`, and it is exact as found: its U and W have entries $0, \pm 1/\sqrt{2}$, which rescaling turns into dyadic coefficients, so no snapping to a dyadic grid is involved. Among the orbit's dyadic orthogonal points, which we enumerate exhaustively, the points minimizing $\gamma_{\infty,\infty}$ also minimize $\gamma_{\infty,2}$, and all of them are equivalent to `4x4x4_48_accurate_262`. The orbit's global $\gamma_{\infty,2}$ minimum, 21.407, is reached only in the other two role assignments and only at points with coefficients in $\mathbb{Q}(\sqrt{2})$, and its measured error is no lower than `4x4x4_48_accurate`'s. The other known rational classes, members of Li, Wang and Hu's curve [13] and Agarwal's scheme [1], have larger $\gamma_{\infty,2}$ minima. So $\gamma_{\infty,2}$ alone is not a sufficient objective for an accuracy search. The rounding-chain factor Q_0 and exact dyadic coefficients matter as much. `4x4x4_48_accurate_262` has dyadic coefficients and the smallest Q_0 and $\gamma_{\infty,\infty}$ of the $\langle 4 \times 4 \times 4 : 48 \rangle$ schemes we measured.

2 Background

2.1 Schemes, growth factors and Q_0

For an $m \times k$ matrix A, $\mathrm{Vect}(A)$ is its row-major vectorization. A bilinear algorithm with r multiplications for $\langle m \times k \times n \rangle$ is a triple of matrices $L \in \mathbb{Q}^{r \times mk}$, $R \in \mathbb{Q}^{r \times kn}$, $P \in \mathbb{Q}^{mn \times r}$ with

$$\mathrm{Vect}(AB) = P\big((L\,\mathrm{Vect}\,A) \odot (R\,\mathrm{Vect}\,B)\big)$$

for all A and B, where $\odot$ is the entrywise product. Here $m = k = n = 4$ and $r = 48$. With L_i the rows of L, R_i the rows of R, $p_{j,i}$ the entries of P and $q^\star$ the dual exponent of q, DPS's (p, q)-growth factor [9, Th. 18] is

$$\gamma_{p,q} = \Big\| \Big(\sum_{i=1}^{r} \|L_i\|_{q^\star} \|R_i\|_{q^\star} \, |p_{j,i}| \Big)_j \Big\|_p ,$$

and the forward error of ℓ recursive levels is at most $f_{p,q}\|A\|_q\|B\|_q\,\varepsilon + O(\varepsilon^2)$ with $f_{p,q} = O(\gamma_{p,q}^{\ell}) = O(n^{\log_4 \gamma_{p,q}})$. The classical algorithm has $\gamma_{\infty,\infty} = \gamma_{\infty,2} = 4$ and $\gamma_{2,\infty} = \gamma_{2,2} = 16$, exponents 1 and 2. DPS also use the smooth bound $\gamma_2 = \sum_i \|L_i\|_2 \|R_i\|_2 \|P_{*,i}\|_2 \geq \gamma_{2,2}$ [8, eq. (5)]. PLinOpt's `growthfactor` tool [11] also reports a combinatorial factor,

$$Q_0 = \max_j \Big(\max_{i:\, p_{j,i} \neq 0} \|L_i\|_0 \, \|R_i\|_0 + \|P_j\|_0 \Big),$$

where $\|\cdot\|_0$ counts nonzeros. It bounds the length of the chain of roundings that feeds one output, and the tool combines it with the growth factors in the constants of its error bounds.

2.2 Isotropies and symmetries

For $(U, V, W) \in GL_4^3$, applying an algorithm to $U^{-1}AV$ and $V^{-1}BW$ returns $U^{-1}ABW$, so

$$(L, R, P) \;\mapsto\; \big(L\,(U^{-1} \otimes V^{\mathsf{T}}),\; R\,(V^{-1} \otimes W^{\mathsf{T}}),\; (U \otimes W^{-\mathsf{T}})\,P\big) \tag{1}$$

is again an algorithm, since $\mathrm{Vect}(XAY) = (X \otimes Y^{\mathsf{T}})\,\mathrm{Vect}\,A$ for row-major vectorization. These are de Groote's isotropies [4]; PLinOpt's orbit tool uses the same group. With $A_i = \mathrm{Mat}(L_i)$, $B_i = \mathrm{Mat}(R_i)$ and Z_i the 4×4 matrix of column i of P, (1) reads

$$A_i \mapsto U^{-\mathsf{T}} A_i V^{\mathsf{T}}, \qquad B_i \mapsto V^{-\mathsf{T}} B_i W^{\mathsf{T}}, \qquad Z_i \mapsto U Z_i W^{-1}.$$

Two further symmetries come from the trace form

$$\textstyle\sum_i \mathrm{tr}(X_i A)\,\mathrm{tr}(Y_i B)\,\mathrm{tr}(Z_i G) = \mathrm{tr}(ABG), \qquad X_i = A_i^{\mathsf{T}},\; Y_i = B_i^{\mathsf{T}} :$$

- the cyclic rotation $(X, Y, Z) \mapsto (Y, Z, X)$;
- the input swap $(X, Y, Z) \mapsto (Y^{\mathsf{T}}, X^{\mathsf{T}}, Z^{\mathsf{T}})$, from $(AB)^{\mathsf{T}} = B^{\mathsf{T}} A^{\mathsf{T}}$.

Every $\gamma_{p,q}$ treats L and R alike, through the norms of their rows, and P differently, through its entries and the norm over outputs. So the input swap leaves the growth factors unchanged, but the rotation does not: the three *role assignments*, one for each factor set that computes the outputs, are three different algorithms for accuracy purposes. γ_2 is symmetric in the three sets, and the rotation leaves it unchanged.

Table 1: Growth factors, γ_2, Q_0 and the operations of the published straight-line programs. In parentheses: the error-bound exponent $\log_k \gamma$ ($k = 2$ for $\langle 2\times2\times2 : 7\rangle$, $k = 4$ for $\langle 4\times4\times4 : 48\rangle$), the power of n with which the error bound grows. Classical multiplication has exponent 1 in the first two columns and 2 in the next two, whose output norm is the 2-norm over all n^2 entries rather than the largest entry. The first five rows are the algorithms of [8, Table 2] (values for `DPS-accurate`, whose coefficients involve $\sqrt{3}$, from that table). The sixth is `4x4x4_48_accurate_262`, in bold where it is the smallest among the $\langle 4 \times 4 \times 4 : 48\rangle$ algorithms with dyadic coefficients. The last two are points of the same orbit from our search and our earlier work [16]. Names are PLinOpt's without the prefixes `2x2x2_7_` and `4x4x4_48_`.

algorithm	$\gamma_{\infty,\infty}$	$\gamma_{\infty,2}$	$\gamma_{2,\infty}$	$\gamma_{2,2}$	γ_2	Q_0	ops
$\langle 2 \times 2 \times 2 : 7\rangle$							
`Winograd`	18 (4.170)	8 (3.000)	31.24 (4.965)	14 (3.807)	17.85	13	15
`Strassen`	12 (3.585)	6.83 (2.772)	17.89 (4.161)	10.45 (3.386)	14.83	8	18
`DPS-accurate` [6]	17.48 (4.128)	5.97 (2.577)	27.71 (4.793)	10.01 (3.323)	–	23	–
$\langle 4 \times 4 \times 4 : 48\rangle$							
`rational` [7]	288 (4.085)	38.16 (2.627)	1032.0 (5.006)	139.9 (3.564)	210.5	152	315
`accurate` [8]	148 (3.605)	25.46 (2.335)	514.9 (4.504)	91.21 (3.256)	154.5	180	284
`accurate_262`	**132** (3.522)	**22.63** (2.250)	**496.5** (4.478)	**90.51** (3.250)	154.5	**80**	262
$\gamma_{\infty,2}$ min. (other roles), $\mathbb{Q}(\sqrt{2})$	176.6 (3.732)	21.41 (2.210)	583.8 (4.595)	85.63 (3.210)	157.3	146	–
`201` [16]	399 (4.320)	75.85 (3.123)	1072.5 (5.033)	225.9 (3.910)	369.7	101	201

2.3 Invariances

Three facts organize the search.

- Scaling one product, $(L_i, R_i, P_{*,i}) \mapsto (\alpha L_i, \beta R_i, P_{*,i}/(\alpha\beta))$, changes no $\gamma_{p,q}$, no γ_2 and not Q_0. So none depends on the scale of U, V or W.
- For orthogonal U, V, W the Kronecker factors in (1) are orthogonal, so the 2-norms of the rows of L and R and of the columns of P are unchanged, and so is γ_2. Then $\gamma_{\infty,2} = \max_{p,q} \sum_i w_i |(UZ_iW^{\mathsf{T}})_{pq}|$ with fixed weights $w_i = \|L_i\|_2 \|R_i\|_2$, and V drops out.
- Multiplying U, V or W on the left by a signed permutation permutes rows and columns of the A_i, B_i, Z_i with signs. That changes no $\gamma_{p,q}$ and not Q_0.

We call the schemes of the orbit, in each role assignment, its *points*. Two points are *equivalent* when scalings of U, V, W, left multiplication of them by signed permutations, a relabelling and rescaling of the products, and the input swap take one to the other. Equivalent points have the same growth factors and the same Q_0, and when the rescalings are by powers of two, a program for one is, after the relabelling, a program for the other with the same roundings in its passes. So equivalent points are one algorithm written in different ways. We name a point after the published scheme it is equivalent to, if any.

3 The 262-operation variant

`4x4x4_48_accurate_262` was added to PLinOpt on 13 October 2026. Its straight-line programs take 64 additions and 8 scalings for L, 67 and 11 for R, and 112 additions and no scalings for P, against `4x4x4_48_accurate`'s 80+4, 68+8 and 108+16. Up to symmetry it is `4x4x4_48_accurate` moved by an orthogonal isotropy with entries $0, \pm 1/\sqrt{2}$. Take `4x4x4_48_accurate`'s (L, R, P) in

its published role and apply (1) with $(U,V,W) = (Q_U, Q_V, Q_W)/\sqrt{2}$, where

$$Q_U = \begin{pmatrix} 0 & -1 & 0 & -1 \\ 1 & 0 & -1 & 0 \\ 0 & -1 & 0 & 1 \\ -1 & 0 & -1 & 0 \end{pmatrix}, \quad Q_V = \begin{pmatrix} -1 & -1 & 0 & 0 \\ -1 & 1 & 0 & 0 \\ 0 & 0 & -1 & -1 \\ 0 & 0 & -1 & 1 \end{pmatrix}, \quad Q_W = \begin{pmatrix} 1 & 0 & -1 & 0 \\ 0 & -1 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}.$$

Each Kronecker factor in (1) is then a $\{0, \pm 1\}$ matrix divided by 2, so the result has coefficients in $\mathbb{Z}[\frac{1}{2}]$ with denominators at most 4, like `4x4x4_48_accurate`. Mapping `4x4x4_48_accurate_262` through the input swap and signed permutations of U, V, W, and relabelling and rescaling the products, gives exactly this scheme.

`4x4x4_48_accurate_262`'s coefficients other than ± 1 are $\pm 1/2$ (224 in L and 80 in R), $\pm 1/4$ (248 in R) and $\pm 3/4$ (8 in R); every entry of P is ± 1. The $\pm 3/4$ belong to the scheme, not to this way of writing it. In eight products one entry of the B side is three times the others; the B side of product 32 in PLinOpt's file, for instance, is

$$\tfrac{1}{4}\,(b_{11} - 3b_{13} - b_{21} - b_{23} + b_{31} + b_{33} + b_{41} + b_{43}).$$

Rescaling a product scales its whole row, so every scheme equivalent to `4x4x4_48_accurate_262` has this ratio. DPS's programs never multiply by 3 or 3/4: their only scalings are 19 halvings, so every coefficient is a sum of terms halved different numbers of times.

Proposition 3.1. *`4x4x4_48_accurate_262` has* $\gamma_{\infty,\infty} = 132$, $\gamma_{\infty,2} = 16\sqrt{2}$, $\gamma_{2,\infty} = 16\sqrt{963} \approx 496.52$, $\gamma_{2,2} = 64\sqrt{2}$, $\gamma_2 = 64(1+\sqrt{2})$ *and* $Q_0 = 80$. *`4x4x4_48_accurate` has* 148, $18\sqrt{2} \approx 25.456$, $8\sqrt{4142} \approx 514.87$, $8\sqrt{130} \approx 91.214$, $64(1+\sqrt{2})$ *and* 180.

The values are exact. Every weight $\|L_i\|_2\|R_i\|_2$ and every column norm of P is a rational or a rational multiple of $\sqrt{2}$, so the sums lie in $\mathbb{Q}(\sqrt{2})$, and we computed them, and the rational squares of $\gamma_{2,\infty}$ and $\gamma_{2,2}$, in exact arithmetic. PLinOpt's `growthfactor`, working in floating point, agrees to every printed digit. Table 1 sets them beside the other algorithms of [8, Table 2] and two points from our search. The error-bound exponents $\log_4 \gamma_{\infty,2}$ and $\log_4 \gamma_{2,2}$ are exactly 9/4 and 13/4. Both algorithms' published programs pass PLinOpt's `SLPchecker` against their matrices, and both pass our exact check against the matrix multiplication tensor.

4 How we found it

4.1 Role assignments

The three role assignments of `4x4x4_48_accurate` have $\gamma_{\infty,2} = 25.456$, 32.971 and 28.142 ($\gamma_{\infty,\infty} =$ 148, 212, 180), so the published one is already the best. For `4x4x4_48_rational` they give 38.162, 39.282 and 33.540, and the published one is not the best. Every search below runs from all three.

4.2 Continuous search over the orbit

We minimize $\gamma_{\infty,2}$ over $(U,V,W) \in GL_4^3$. The max over the 16 outputs is smoothed by a log-sum-exp with sharpness β and $|x|$ by $\sqrt{x^2+\epsilon^2}$, the smoothed objective is minimized by BFGS with a backtracking line search, and (β, ϵ) runs through a continuation from $(0.5, 10^{-1})$ to $(32768, 10^{-6})$. After each stage each of U, V, W is rescaled to determinant ± 1, which changes nothing (Section 2.3).

The gradient is exact: with $M_i = U^{-\mathsf{T}} A_i V^{\mathsf{T}}$, $N_i = V^{-\mathsf{T}} B_i W^{\mathsf{T}}$, $Z'_i = U Z_i W^{-1}$, $a_i = \|M_i\|_F$, $b_i = \|N_i\|_F$ and π the softmax weights of the outputs, the adjoints are

$$\frac{\partial f}{\partial Z'_i} = \pi \circ a_i b_i \, s'(Z'_i), \qquad \frac{\partial f}{\partial M_i} = \frac{b_i S_i}{a_i} M_i, \qquad \frac{\partial f}{\partial N_i} = \frac{a_i S_i}{b_i} N_i,$$

with $S_i = \sum_{pq} \pi_{pq} s(Z'_{i,pq})$. They are carried through the products and through each inverse by $\partial X = -X \, \partial M \, X$. The implementation (`gamma48`, in Rust) checks this gradient against central differences to 10^{-10} relative. It runs on all cores, and one start takes about 0.5 s on one core. Starts are the identity, random near-identity points, and random rotations times random perturbations of the identity.

Within the orthogonal sub-orbit (γ_2 fixed at its minimum 154.51; 12 parameters, two Cayley-parametrized rotations), the minimum is $16\sqrt{2} \approx 22.627$ for the published role and $12 + 8\sqrt{2} \approx 23.314$ for the other two. At the minimum U and W have entries $0, \pm 1/\sqrt{2}$ to machine precision, so the optimum is an exact point. Rescaling the products makes its coefficients dyadic (Section 4.3), and it is equivalent to `4x4x4_48_accurate_262`. Over the whole orbit, 30,000 starts from `4x4x4_48_accurate` (10,000 per role) and 7,200 each from `4x4x4_48_rational` and from `4x4x4_48_201` give the same local optima, a direct sign that the three lie in one orbit. The role decides what the search finds:

- In the published role the lowest optimum is $16\sqrt{2}$, reached 248 times in `4x4x4_48_accurate`'s 10,000 starts, and nothing lower appears. The lowest point the search finds in this role is equivalent to `4x4x4_48_accurate_262`, and exact as found: we did not reach it by snapping a lower point to a dyadic grid.

- In the other two roles the lowest optimum, 21.4073, is found thousands of times, and nothing lower appears; the next is 21.944. Its points have two orthogonal factors and one with singular values $(1, 1, s, s)$, $s = 2(\sqrt{2} - 1)$, so their coefficients lie in $\mathbb{Q}(\sqrt{2})$. They cannot be made dyadic: the scaling enters R as s and P as $1/s$, and no dyadic value other than a power of two has a dyadic inverse. Snapping them to the dyadic orthogonal points of their roles would not help either: those have $\gamma_{\infty,2} \geq 12 + 8\sqrt{2}$ (Section 4.3), more than `4x4x4_48_accurate_262`'s $16\sqrt{2}$.

4.3 The dyadic orthogonal sub-orbit, exhaustively

A scheme is exact in binary floating point only if its coefficients are dyadic, and a scaling is exact only if it is by a power of two: multiplying by another dyadic constant, such as 3/4, rounds. `4x4x4_48_accurate_262`'s R has eight coefficients $\pm 3/4$ and `4x4x4_48_accurate`'s has sixteen, but DPS's programs for them never multiply by 3/4. They build it from halvings and additions, so all their rounding is in additions. Isotropies with dyadic entries and dyadic inverses keep a scheme dyadic, and up to scale they are dense in $\mathrm{GL}_4(\mathbb{R})$, since they include the elementary matrices $I + tE_{ij}$ for every dyadic t. So dyadic points come arbitrarily close to the $\mathbb{Q}(\sqrt{2})$ minimum, but only with coefficients of ever more bits, and multiplying by those rounds just as multiplying by s does. We look instead for points with small dyadic coefficients. Let G be the set of orthogonal 4×4 matrices whose entries lie in $\mathbb{Z}[\frac{1}{2}]$ or in $\mathbb{Z}[\frac{1}{2}]/\sqrt{2}$. It has 2304 elements:

- the 384 signed permutations and the 768 matrices with entries $\pm 1/2$, which together form the Weyl group $W(F_4)$;

- the 1152 matrices with entries $0, \pm 1/\sqrt{2}$.

Table 2: Smallest $\gamma_{\infty,2}$ found over the whole orbit of each rational class, 2400 starts per role, and the Li–Wang–Hu invariant φ. The role numbering follows each scheme's own coordinates.

class	φ	role 0	role 1	role 2	minimum
$s(1/4)$, the DPS class	192	21.407	22.627	21.407	21.407
$s(1/2)$	336	23.144	23.654	23.144	23.144
$s(1)$	1092	25.167	26.833	25.167	25.167
$s(2)$	4161	27.852	32.485	27.852	27.852
$s(4)$	16448.25	32.677	40.319	32.677	32.677
$s(8)$	65600.06	40.134	51.351	40.134	40.134
Agarwal's scheme	–	39.619	42.924	39.619	39.619

It is closed under products and inverses. For $(U, V, W) \in G^3$ the three Kronecker factors of (1) carry the factors $\sqrt{2}^{-(c_U+c_V)}$, $\sqrt{2}^{-(c_V+c_W)}$ and $\sqrt{2}^{-(c_U+c_W)}$, with $c = 1$ for the $1/\sqrt{2}$ matrices and 0 otherwise. Their product is a power of two, so scaling each product makes every coefficient dyadic. By Section 2.3 the growth factors and Q_0 depend only on the classes of U, V, W under left multiplication by signed permutations. There are six classes of 384, so the sub-orbit of each role has 216 distinct points, 648 in all, and we enumerate them.

Theorem 4.1. *Among the* 648 *points of the dyadic orthogonal sub-orbit of* `4x4x4_48_accurate`*'s scheme, the minimum of* $\gamma_{\infty,2}$ *is* $16\sqrt{2}$*, attained at* 42 *points, all in the published role; in each of the other two roles it is* $12 + 8\sqrt{2}$*, attained at* 36 *points. The minimum of* $\gamma_{\infty,\infty}$ *is* 132*, attained at exactly eight points, all in the published role, and these eight also attain* $\gamma_{\infty,2} = 16\sqrt{2}$*. Each of the eight is equivalent to* `4x4x4_48_accurate_262`*.*

The proof is the enumeration together with an explicit equivalence for each of the eight points (`equiv.py`). `4x4x4_48_accurate`'s own point has the fourth smallest $\gamma_{\infty,2}$ in the sub-orbit (25.456, after $16\sqrt{2}$, $12 + 8\sqrt{2}$ and 24). So within the dyadic orthogonal moves of `4x4x4_48_accurate`, the points that are best for both growth factors are all equivalent to `4x4x4_48_accurate_262`.

4.4 Other classes

Rational rank-48 schemes are not all in one orbit. Li, Wang and Hu put DPS's scheme on a curve $s(t)$ whose members are pairwise inequivalent for $t \geq 1/4$ [13], with the invariant $\varphi = 64 + 4/t^2 + 1024t^2$; the DPS class is $t = 1/4$, the minimum of φ. For $t = 2^k$ the coefficients are dyadic. Agarwal replaced twelve products of $s(2)$ to get a scheme off the curve [1]. We rebuilt both exactly from the papers' statements (checked on all 4096 Brent equations) and ran the continuous search from each, 2400 starts per role (Table 2). The smallest $\gamma_{\infty,2}$ grows steadily with t. No other class has any point, dyadic or not, as low as `4x4x4_48_accurate_262`'s 22.63; the closest, $s(1/2)$, bottoms out at 23.14.

5 Accuracy

5.1 Implementation

The measurements use our fast matrix multiplication library for Apple silicon [17]. One level of a $\langle 4 \times 4 \times 4 : 48 \rangle$ scheme runs as three fused passes, each one streaming loop over the elements of the blocks:

- the left pass computes all 48 combinations $L \operatorname{Vect} A$ of the 16 blocks of A;

- the right pass does the same for B;
- the output pass combines the 48 block products into the 16 blocks of C.

The block products go to Accelerate's GEMM (`cblas_sgemm`, `cblas_dgemm`). The passes are generated from the published straight-line programs, operation by operation in the programs' order, so each program's rounding is the one measured. Deeper levels recurse on the block products. One, two and three levels at $n = 4096$ leave GEMMs of size 1024, 256 and 64, and at $n = 8192$ sizes 2048, 512 and 128. For comparison we include `2x2x2_7_Winograd` (Strassen–Winograd, 15 additions) at one, two and four levels. Two of its levels reduce the size as much as one $\langle 4 \times 4 \times 4 \rangle$ level.

5.2 Protocol

- Inputs are uniform on $[-1, 1)$ or standard normal, from a xorshift generator. Each setting uses five seeds, three for double precision at $n = 8192$.
- Single precision is measured against the double-precision product of the same inputs (Accelerate's `dgemm`). Its 29 extra bits make it exact for this purpose.
- Double precision is measured against a compensated product: every entry is an Ogita–Rump–Oishi Dot2 dot product [15], with each product split exactly by a fused multiply-add and the sum accumulated by error-free TwoSum steps, then rounded to double. Its own error is about the unit roundoff, two orders of magnitude below the errors measured.
- The error is $\max_{ij}|\hat{C}_{ij} - C_{ij}| / \max_{ij}|C_{ij}|$. We report it as a multiple of the classical GEMM's own error on the same inputs.

The programs are:

- `4x4x4_48_accurate` with DPS's 284-operation programs;
- `4x4x4_48_accurate_262` with DPS's 262-operation programs;
- `4x4x4_48_accurate_262` with our programs. Our scheme is equivalent to `4x4x4_48_accurate_262`: the input swap, signed permutations and a rescaling of the products take one to the other. Ours has its constants $\pm 1/2$ and $\pm 1/4$ moved into P: no entry of its P is ± 1, and its L has ± 2, ± 3, $\pm 3/2$ and $\pm 1/2$. Its programs come from one default run of PLinOpt's `optimizer`: 445 operations. The optimizer is randomized, and a rerun gives 437. The same run on DPS's scheme gives 383, so how the scheme is written accounts for about 60 of the 180 operations between our programs and DPS's;
- `4x4x4_48_rational`'s 315-operation programs and the 201-operation programs of [16];
- the orbit's $\gamma_{\infty,2}$ minimum, in another role, written as direct sums with its coefficients rounded to the working precision;
- `4x4x4_48_accurate` written as direct sums, a control for program style; unlike DPS's programs, these multiply by 3/4 directly;
- two dyadic points of `4x4x4_48_accurate`'s orbit, written as direct sums, chosen from the enumeration of Section 4.3 to separate the two growth factors. Probe A has `4x4x4_48_accurate_262`'s $\gamma_{\infty,2} = 16\sqrt{2}$ but $\gamma_{\infty,\infty} = 256$ (role 0, classes $5, 2, 3$); probe B has $\gamma_{\infty,2} = 28.97$, larger than `4x4x4_48_accurate`'s, but $\gamma_{\infty,\infty} = 144$, smaller (role 2, classes $4, 4, 2$). Their Q_0 are 144 and 94, and their coefficients are ± 1, $\pm 1/2$ and $\pm 1/4$, so their direct sums scale only by powers of two.

Table 3: Error against the exact reference as a multiple of the classical GEMM's own error, mean over all instances: single precision 20 instances ($n = 4096$ and 8192, uniform and normal, five seeds each); double precision 16 for the first five schemes ($n = 4096$, five seeds per distribution; $n = 8192$, three) and 10 ($n = 4096$ only) for the rest. Winograd's levels are binary, so its columns are two and four levels; "–": not measured.

	single precision			double precision		
levels	1	2	3	1	2	3
`accurate`, DPS's programs (284)	0.94	1.05	1.54	0.94	1.03	1.57
`accurate_262`, DPS's programs (262)	0.73	0.79	1.17	0.72	0.81	1.16
`accurate_262`, our programs (445)	0.71	0.83	1.33	0.73	0.83	1.32
`accurate`, direct sums	0.96	1.07	1.65	0.93	1.08	1.70
probe A, direct sums	0.74	0.82	1.32	0.71	0.85	1.36
probe B, direct sums	0.78	0.82	1.26	0.74	0.86	1.36
orbit's $\gamma_{\infty,2}$ minimum (other roles), $\mathbb{Q}(\sqrt{2})$	0.86	1.00	1.61	0.88	1.01	1.57
`rational`, DPS's programs (315)	1.06	1.60	3.27	1.03	1.56	3.23
`201`	2.78	8.72	36.84	2.59	8.74	36.77
`2x2x2_7_Winograd`, 2 and 4 binary levels	–	3.11	10.25	–	3.00	9.93

5.3 Results

Table 3 gives each scheme's mean error as a multiple of the classical GEMM's. Table 4 gives `4x4x4_48_accurate_262`'s error divided by `4x4x4_48_accurate`'s on the same inputs, and Figure 1 plots the means.

- `4x4x4_48_accurate_262` with DPS's programs is more accurate than `4x4x4_48_accurate` at every depth, in both precisions and for both distributions. Its error is 0.73–0.83 of `4x4x4_48_accurate`'s averaged over the seeds of each setting, 0.75–0.77 averaged over all instances, and 0.62–0.90 for single instances. It has the lowest error of all at two and three levels. At one level `4x4x4_48_accurate_262` with our programs (0.71 in single precision) and probe A (0.71 in double) are slightly lower than its 0.73 and 0.72.
- At one and two levels, where the library's plans run, it is more accurate than the classical GEMM itself (0.72–0.81 of its error), while `4x4x4_48_accurate` is about as accurate as the classical GEMM (0.94–1.05). The block products are dot products a quarter or a sixteenth as long, which saves more than the passes add.
- Single and double precision agree within a few percent, and so do the two sizes and the two distributions.
- Over these depths the advantage is nearly constant: from two levels to three both errors grow by a factor of 1.46–1.47 in single precision (1.52 for `4x4x4_48_accurate` and 1.43 for the variant in double). With large leaves the leaf GEMMs' error dominates, and the difference comes from the passes' constant (Q_0), not yet from the growth factor. Section 5.4 recurses down to scalars to see the growth.

5.4 Deep recursion

To see the growth factors at work we recursed down to scalars: $n = 1024$ with one to five levels (leaves 256, 64, 16, 4 and 1), as in DPS's experiments [8, Fig. 1], ten instances per precision. Table 7

Table 4: Error of `accurate_262` (DPS's programs) divided by `accurate`'s on the same inputs: mean over the seeds, with the range in brackets.

precision	setting	1 level	2 levels	3 levels
single	$n = 4096$, uniform	0.82 [0.75, 0.90]	0.74 [0.67, 0.81]	0.78 [0.76, 0.82]
single	$n = 4096$, normal	0.76 [0.64, 0.88]	0.81 [0.77, 0.84]	0.75 [0.66, 0.80]
single	$n = 8192$, uniform	0.77 [0.69, 0.85]	0.74 [0.67, 0.79]	0.76 [0.70, 0.78]
single	$n = 8192$, normal	0.77 [0.73, 0.81]	0.73 [0.62, 0.77]	0.75 [0.68, 0.85]
double	$n = 4096$, uniform	0.79 [0.73, 0.86]	0.78 [0.72, 0.83]	0.73 [0.70, 0.82]
double	$n = 4096$, normal	0.77 [0.71, 0.83]	0.78 [0.75, 0.85]	0.76 [0.67, 0.89]
double	$n = 8192$, uniform	0.73 [0.64, 0.83]	0.83 [0.80, 0.88]	0.74 [0.74, 0.74]
double	$n = 8192$, normal	0.75 [0.68, 0.81]	0.78 [0.75, 0.81]	0.76 [0.70, 0.83]

Table 5: The factors beside the measured single-precision error at three levels (a multiple of the classical GEMM's error, mean over all instances) and its growth from two levels to three. Sorted by the measured error.

scheme	$\gamma_{\infty,2}$	$\gamma_{\infty,\infty}$	Q_0	program	3 levels	growth
`accurate_262`	22.63	132	80	DPS, 262	1.17	1.47
probe B	28.97	144	94	direct sums	1.26	1.53
probe A	22.63	256	144	direct sums	1.32	1.62
`accurate_262`	22.63	132	80	ours, 445	1.33	1.60
`accurate`	25.46	148	180	DPS, 284	1.54	1.46
orbit's $\gamma_{\infty,2}$ min. (other roles)	21.41	176.6	146	direct sums	1.61	1.61
`accurate`	25.46	148	180	direct sums	1.65	1.55
`rational`	38.16	288	152	DPS, 315	3.27	2.05
`201`	75.85	399	101	ours, 201	36.84	4.22

gives the results and Figure 2 plots them.

- `4x4x4_48_accurate_262` with DPS's programs is again the most accurate from two levels on, and at one level in double precision; at one level in single precision probe A and our programs for `4x4x4_48_accurate_262` have 0.78 against its 0.79. Its advantage over `4x4x4_48_accurate` grows with depth: its error is 0.80–0.83 of `4x4x4_48_accurate`'s at one level and 0.72 at five (single instances 0.67–0.80), in both precisions.

- From four levels to five, the errors grow by 2.60–2.61 for `4x4x4_48_accurate_262` and 2.75–2.79 for `4x4x4_48_accurate`, about 6% slower per level. That is the direction $\gamma_{\infty,2}$ predicts, but less than its ratio $16\sqrt{2}/18\sqrt{2} = 8/9$ would give.

- Programs matter more at depth. `4x4x4_48_accurate_262` with our 445-operation programs and the two probes beat `4x4x4_48_accurate` in the main campaign, where the leaves are larger, but at $n = 1024$ they fall behind `4x4x4_48_accurate` with DPS's programs from three levels on: 2.12–2.29 against 2.00 at three levels and 13.96–15.45 against 13.29 at five, in single precision. Only `4x4x4_48_accurate_262` with DPS's programs beats `4x4x4_48_accurate` at every depth. Section 7 explains why.

- Among the direct-sum programs, the dyadic probes stay 24–29% below `4x4x4_48_accurate`'s at five levels and the orbit's $\gamma_{\infty,2}$ minimum only about 8%: the rounding of its constants compounds with depth.

Table 6: Time of DPS's programs in our library: the best of three alternating rounds, each the median of five runs.

precision	plan, n	`accurate`	`accurate_262`	ratio
single	`D48/G`, 4096	0.044 s	0.044 s	1.000
single	`D48/G6`, 4096	0.036 s	0.036 s	1.000
single	`D48+4/D48/G6`, 8192	0.236 s	0.237 s	1.004
single	`D48/G`, 8192	0.272 s	0.271 s	0.996
double	`D48/G`, 4096	0.139 s	0.140 s	1.007
double	`D48/G6`, 4096	0.177 s	0.173 s	0.977
double	`D48+4/D48/G6`, 8192	0.800 s	0.803 s	1.004
double	`D48/G`, 8192	1.054 s	1.061 s	1.007

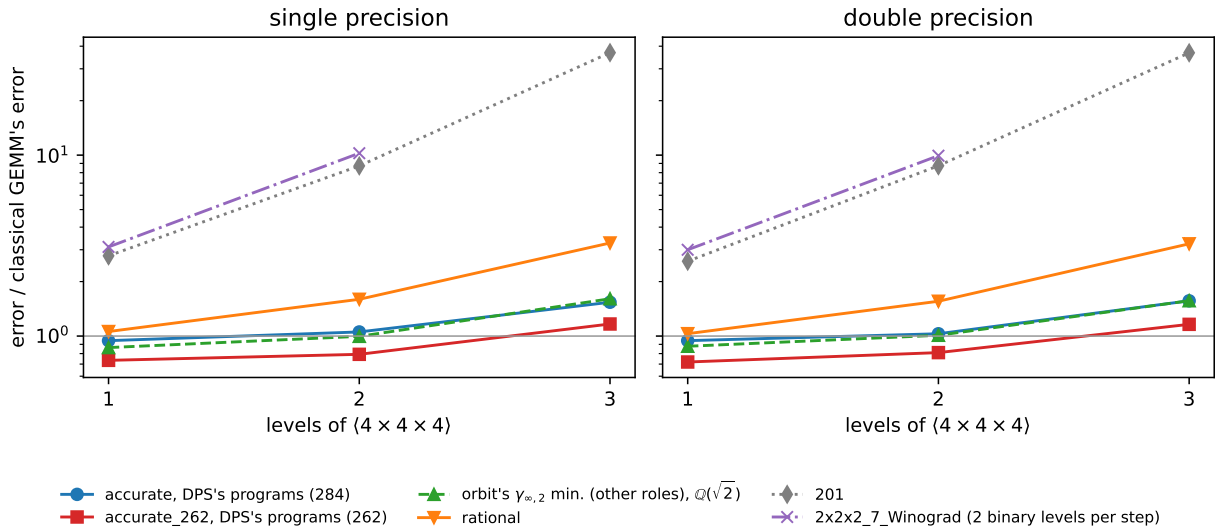


Figure 1: Mean error against the exact reference as a multiple of the classical GEMM's error, over all sizes, distributions and seeds of the main campaign ($n = 4096$ and 8192). The points for `2x2x2_7_Winograd` are two and four binary levels.

- `4x4x4_48_rational` grows by about 3.6 per level, `4x4x4_48_201` by about 7.5, and `2x2x2_7_Winograd` by about 7 per two binary levels.

6 Speed

Table 6 times DPS's programs in our library. It covers single levels with one GEMM at a time (`D48/G`) and the plans the library picks at these sizes, which use six GEMM streams and pipelined passes (`D48/G6`, `D48+4/D48/G6`), in alternating rounds. Both take the same time to within the timer's resolution (0.977–1.007). The passes move the same data, and with 262 against 284 operations neither is bound by arithmetic. Our 445-operation programs for `4x4x4_48_accurate_262` also took `4x4x4_48_accurate`'s time. In an earlier run of three alternating rounds of five, their fastest run was 0.998–1.007 times `4x4x4_48_accurate`'s in seven of the eight configurations, and 1.041 in the eighth (double precision, `D48/G6`, $n = 4096$), where the medians of the rounds vary by up to 10%. Their passes are slower, 0.057 against 0.054–0.055 s at $n = 8192$ in double precision,

Table 7: Deep recursion at $n = 1024$: mean error against the exact reference as a multiple of the classical GEMM's error, over ten instances per precision (uniform and normal, five seeds each); five levels reach scalar leaves. In parentheses: operation counts of DPS's programs, or of ours where marked. Winograd: twice as many binary levels.

	single precision					double precision				
levels	1	2	3	4	5	1	2	3	4	5
`accurate` (284)	0.95	1.10	2.00	4.77	13.29	0.91	1.18	2.09	4.91	13.50
`accurate_262` (262)	0.79	0.97	1.67	3.75	9.79	0.76	0.98	1.63	3.68	9.58
`accurate_262`, ours (445)	0.78	1.10	2.29	5.71	15.45	0.78	1.09	2.20	5.48	15.62
`accurate`, direct	1.00	1.35	2.69	6.76	19.63	0.95	1.29	2.57	6.62	19.36
probe A	0.78	1.08	2.27	5.40	14.91	0.80	1.09	2.21	5.43	14.45
probe B	0.82	1.08	2.12	5.28	13.96	0.82	1.05	2.12	5.10	14.05
$\gamma_{\infty,2}$ min. (other roles), $\mathbb{Q}(\sqrt{2})$	0.89	1.22	2.40	6.42	18.06	0.92	1.20	2.36	5.89	17.57
`rational`	1.08	1.92	4.76	15.48	56.64	1.13	1.90	4.80	15.36	55.20
`201`	2.75	9.86	48.87	304	2242	2.89	9.60	47.50	301	2293
`Winograd`	2.88	10.46	46.34	254	1726	3.38	11.14	47.75	248	1753

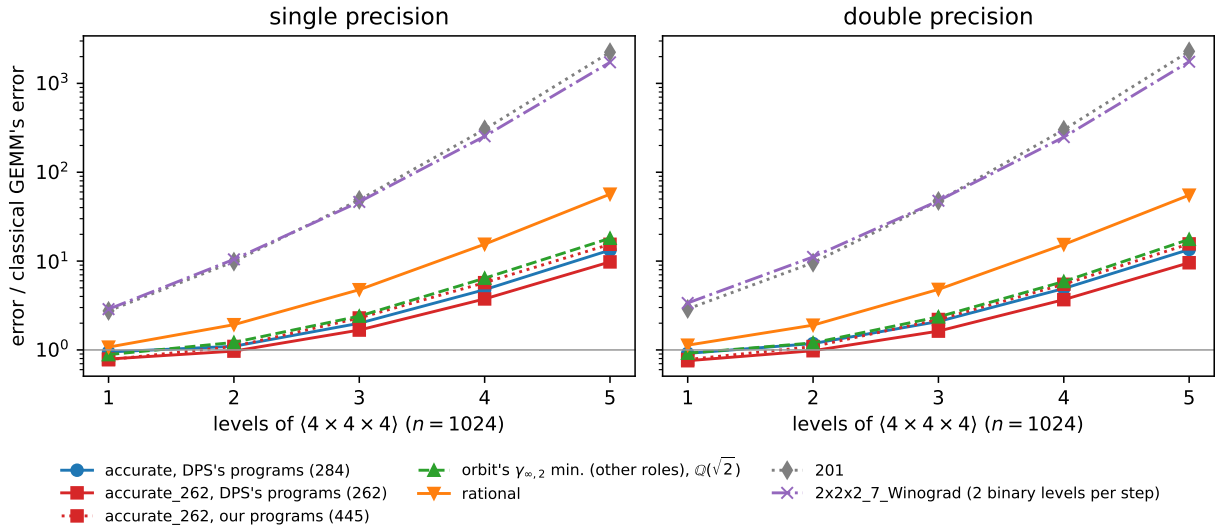


Figure 2: Deep recursion at $n = 1024$: mean error against the exact reference as a multiple of the classical GEMM's error, over ten instances per precision. The points for `2x2x2_7_Winograd` are two binary levels per step.

but the passes take only 5–17% of the time. So the more accurate algorithm costs nothing at run time, and it is the cheaper one in operation count.

7 Discussion

What predicts the measured error. Table 5 sets the factors beside the error measured at three levels in single precision, where the schemes differ most. The growth factors separate the distant schemes (`4x4x4_48_rational`, `4x4x4_48_201`, two levels of `2x2x2_7_Winograd`), but no single factor orders the close ones.

Table 8: The first-order rounding model of one level of each program (Section 7) beside the measured errors. L+R and P: the RMS error those passes add, mean over the outputs; worst: all passes, worst output; all three relative to an output's RMS, in units of one rounding. Count and size: the roundings of the P pass that reach an output and their RMS size in output units (value times weight), means over the outputs; an output's P error is $\sqrt{\text{count}}$ times the size. Amp.: how much the worst output amplifies the leaf products' own errors, a property of the scheme. Measured: single-precision error as a multiple of the classical GEMM's at three levels (Table 3) and five (Table 7).

	model, one level						measured	
program	L+R	P	count	size	worst	amp.	3 levels	5 levels
`accurate_262`, DPS's programs (262)	5.74	3.09	15.0	0.80	7.22	2.83	1.17	9.79
`accurate_262`, our programs (445)	5.61	8.22	35.6	1.39	12.51	2.83	1.33	15.45
probe B, direct sums	7.19	7.42	29.0	1.38	10.75	2.83	1.26	13.96
probe A, direct sums	10.12	5.67	15.0	1.46	11.77	2.83	1.32	14.91
`accurate`, DPS's programs (284)	5.57	3.35	18.0	0.79	7.12	3.16	1.54	13.29
`accurate`, direct sums	7.64	5.98	18.0	1.41	11.14	3.16	1.65	19.63
`rational`, DPS's programs (315)	7.75	7.11	47.8	1.04	11.73	4.00	3.27	56.64
`201`, our programs (201)	12.30	8.87	22.7	1.85	21.05	9.53	36.84	2242

- Probe A has `4x4x4_48_accurate_262`'s $\gamma_{\infty,2}$ but a larger measured error.
- Probe B has a larger $\gamma_{\infty,2}$ than `4x4x4_48_accurate` but a smaller measured error.
- The orbit's $\gamma_{\infty,2}$ minimum, in another role, has the smallest $\gamma_{\infty,2}$ of all but about the same measured error as `4x4x4_48_accurate`.

Among the dyadic schemes, Q_0 comes closest to the order of the measured errors: 80, 94, 144 and 180 for `4x4x4_48_accurate_262`, probe B, probe A and `4x4x4_48_accurate`. Of the dyadic schemes, `4x4x4_48_accurate_262` is best or tied for best in every factor, and with DPS's programs its measured error is the smallest.

Dyadic coefficients. The orbit's $\gamma_{\infty,2}$ minimum and probe A have nearly the same $\gamma_{\infty,2}$ (21.41 and 22.63) and Q_0 (146 and 144), and the former has the smaller $\gamma_{\infty,\infty}$ (176.6 against 256). Yet the former's measured error is about 20% larger at three levels: 1.61 against 1.32 times the classical error in single precision, and 1.57 against 1.36 in double. Its constants, such as $2(\sqrt{2}-1)$, are rounded to the working precision, so the scheme itself is inexact by a unit roundoff, and each multiplication by them rounds. Multiplications by $\pm 1/2^k$ are exact. The gap compounds with depth: at five levels the probes are 24–29% below `4x4x4_48_accurate`'s direct sums and the $\mathbb{Q}(\sqrt{2})$ point only 8%. A search for accuracy should therefore keep to small dyadic coefficients, and its programs to scalings by powers of two.

Programs. The same scheme has different measured errors with different programs. `4x4x4_48_accurate_262` with DPS's programs beats `4x4x4_48_accurate_262` with ours, and `4x4x4_48_accurate` with DPS's programs beats `4x4x4_48_accurate` as direct sums (1.17 against 1.33, and 1.54 against 1.65, times the classical error at three levels in single precision). Our scheme for `4x4x4_48_accurate_262` is equivalent to DPS's by relabelling and by scalings by powers of two, which are exact in floating point, so the first difference lies entirely in the programs. It is not their length. DPS's programs have the longer chains of roundings, up to 22 through one level against 18 for ours, and the probes' direct sums have 37 and 39 yet beat our programs (Table 8). Nor is

the second difference the direct sums' multiplications by 3/4: in the model below they change the result by under 1%.

A rounding model. Table 8 models one level of each program to first order. Every addition, and every scaling by a constant that is not a power of two, rounds, with an error proportional to the exact value rounded, and the errors are independent. The inputs are independent with unit variance, so the products have covariance $(L_i \cdot L_{i'})(R_i \cdot R_{i'})$ and the size of every intermediate value, cancellation included, is an exact second moment. Propagating each rounding to the outputs gives the error each pass adds to a level.

- For `4x4x4_48_accurate_262`, DPS's programs and ours add about the same error in the L and R passes (5.74 against 5.61) but not in the P pass (3.09 against 8.22). The sixteen products that feed an output are each $1/\sqrt{2}$ of its size and would sum to $2\sqrt{2}$ times its size if they did not cancel. DPS's P pass keeps its partial sums small: each output collects 15 roundings whose RMS size is 0.80 of the output's. Ours lets them grow: each output collects 36 roundings, since our program shares more partial sums between outputs, of RMS size 1.39.
- At one level the leaf products' own errors dominate, and the two programs measure the same. Every further level amplifies those by the same factor and adds a new pass error, so the gap grows with depth (Table 7).
- For the four programs whose schemes amplify leaf errors equally ($2\sqrt{2}$ for the worst output), the model's worst-output error orders the measured errors exactly, at three levels and at five.
- `4x4x4_48_accurate`, `4x4x4_48_rational` and `4x4x4_48_201` amplify leaf errors more (3.16, 4 and 9.53). `4x4x4_48_accurate`'s own programs add no more error than `4x4x4_48_accurate_262`'s (7.12 against 7.22); its larger measured error comes from its scheme's larger amplification. The same trade-off explains the crossover of Section 5.4: the probes and our programs for `4x4x4_48_accurate_262` amplify less than `4x4x4_48_accurate` but add more error per level (10.75–12.51 against 7.12), so they beat `4x4x4_48_accurate` while the leaves' own errors dominate and fall behind it at depth.

Recommendation. Wherever `4x4x4_48_accurate` is used, `4x4x4_48_accurate_262` is better on both counts: 22 fewer operations and about a quarter less error, in the same orbit and with the same γ_2.

Limitations.

- The measurements come from one machine and one implementation style: fused passes over a vendor GEMM, with large leaves. The inputs are i.i.d. uniform or normal matrices, the error is the max-norm error relative to $\max|C|$, and there are at most three levels at $n = 4096$ and 8192 and five at $n = 1024$. Structured or badly scaled inputs may rank the schemes differently.
- The continuous search gives no proof of global optimality. Only the dyadic orthogonal moves are enumerated exhaustively; non-orthogonal dyadic isotropies, such as shears and scalings by two, are not.

Outlook. The natural next objective combines the growth factors with Q_0 and the operation count over dyadic points, or uses the measured error of deep recursion directly; `gamma48` makes such searches cheap. PLinOpt's optimizer minimizes operations, and the rounding model gives a second objective, the error a program adds; it evaluates a program in well under a second. DPS's P programs already keep their partial sums small, and an optimizer that also limits how many roundings reach each output might give `4x4x4_48_accurate_262` programs more accurate than DPS's. Lower precisions (bf16, fp16), where every level's error is a larger share of the result, are where an accurate scheme matters most.

8 Data and code

Everything is in the public repository https://github.com/gsidebottom/logic; paths below are relative to it.

- The search: `src/bin/gamma48.rs` (the continuous search) and `matmul/dps48/gamma/`, which holds the role assignments and the isotropy action (`orbit.py`), the earlier Python search (`search_orth.py`, `search_gl.py`), the exact constructions (`exact.py`), the exhaustive dyadic enumeration (`dyadic_orth.py`) and the equivalence test (`equiv.py`, `equiv262.py`).
- Growth factors: `matmul/fastmm/growth.py`, checked against PLinOpt's `growthfactor`.
- Other classes: `matmul/dps48/lwh_family.py` and `agarwal_replacement.py`, which rebuild the Li–Wang–Hu members and Agarwal's scheme from the papers.
- Measurements: `matmul/fastmm` (`bench_fmm.c`, `bench_impl.h`, `gen_kernels.py`). The builds are `build/bench_fmm_acc262` for the 262-operation programs and `build/bench_fmm_acc284` for `4x4x4_48_accurate`'s. The runs labelled `bench_fmm` in the result files used `4x4x4_48_accurate`'s programs, the library's default until it switched to `4x4x4_48_accurate_262` after these measurements. `FMM_EXACT`, `FMM_NORMAL` and `FMM_SEED` select the reference, the distribution and the instance, and `FMM_REFCACHE` shares references between builds. The probes' programs are in `matmul/dps48/gamma/slp/`. Raw results are in `matmul/fastmm/results/` (`accuracy_campaign_2026-10-25.txt`, `accuracy_deep_2026-10-25.txt`, `gamma_search_2026-10-25.txt`, `gamma_classes_2026-10-25.txt`), and `matmul/fastmm/paper_acc262.py` turns them into this paper's tables and figures.
- The rounding model: `matmul/dps48/gamma/slp_errmodel.py`, with its output in `matmul/fastmm/results/slp_errmodel_2026-10-25.txt`; `paper_acc262.py--model` writes Table 8.
- The schemes: PLinOpt's `data/4x4x4_48_accurate_262` and `data/4x4x4_48_accurate` [11]; our scheme equivalent to `4x4x4_48_accurate_262` is `matmul/dps48/gamma/g16s2_{L,R,P}.sms` and our programs for it `matmul/dps48/gamma/slp/g16s2o_{L,R,P}.slp`.

Acknowledgments

This work was carried out in an interactive collaboration with Claude (Anthropic; the Opus 5.5 model), which implemented the search, measurement and verification tools and much of this text under the author's direction and review. Every claim is checked mechanically by the tools listed above; the results rest on those checks rather than on trust in either the author or the tools. The

algorithm comes from Dumas, Pernet and Sedoglavic's work and was published by J.-G. Dumas in PLinOpt; our search found a scheme equivalent to it, and we learned that it was already there only after constructing ours.

References

[1] A. Agarwal, *Certified local rank and uniqueness barriers for a 48-term matrix-multiplication decomposition*, arXiv:2609.13596 (2026).

[2] D. Bini, G. Lotti, *Stability of fast algorithms for matrix multiplication*, Numer. Math. 36 (1980) 63–72.

[3] R. P. Brent, *Algorithms for matrix multiplication*, Technical Report STAN-CS-70-157, Stanford University (1970).

[4] H. F. de Groote, *On varieties of optimal algorithms for the computation of bilinear mappings I. The isotropy group of a bilinear mapping*, Theoret. Comput. Sci. 7 (1978) 1–24.

[5] J. Demmel, I. Dumitriu, O. Holtz, R. Kleinberg, *Fast matrix multiplication is stable*, Numer. Math. 106 (2007) 199–224.

[6] J.-G. Dumas, C. Pernet, A. Sedoglavic, *Strassen's algorithm is not optimally accurate*, Proc. ISSAC 2024, 254–263.

[7] J.-G. Dumas, C. Pernet, A. Sedoglavic, *A non-commutative algorithm for multiplying* 4×4 *matrices using* 48 *non-complex multiplications*, arXiv:2506.13242 (2025).

[8] J.-G. Dumas, C. Pernet, A. Sedoglavic, *A more accurate rational non-commutative algorithm for multiplying* 4×4 *matrices using* 48 *multiplications*, arXiv:2603.18699 (v1 19 March 2026, v3 29 July 2026).

[9] J.-G. Dumas, C. Pernet, A. Sedoglavic, *Towards automated generation of fast and accurate algorithms for recursive matrix multiplication*, J. Symbolic Comput. 134 (2026) 102524.

[10] J.-G. Dumas, C. Pernet, A. Sedoglavic, J. Stapleton, P. Tichavský, *Fast matrix multiplication via recursive* $\langle 4\times4\times4 : 48\rangle$ *algorithms into practice*, arXiv:2609.12027 (v1 10 September 2026, v2 2 October 2026).

[11] J.-G. Dumas, B. Grenet, C. Pernet, A. Sedoglavic, *PLinOpt, a collection of C++ routines handling linear & bilinear programs*, https://github.com/jgdumas/plinopt; data files `4x4x4_48_accurate` (commit `fb753b5`, 23 June 2026) and `4x4x4_48_accurate_262` (commit `b441528`, 13 October 2026).

[12] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed., SIAM, 2002.

[13] X. Li, Y. Wang, S. Hu, *Substitution and quotient of the isotropy group action*, arXiv:2607.15069 (2026).

[14] A. Novikov et al., *AlphaEvolve: a coding agent for scientific and algorithmic discovery*, arXiv:2506.13131 (2025).

[15] T. Ogita, S. M. Rump, S. Oishi, *Accurate sum and dot product*, SIAM J. Sci. Comput. 26 (2005) 1955–1988.

[16] G. Sidebottom, *201 operations for rank-48* 4×4 *matrix multiplication over rings containing* $1/2$, preprint (2026); PLinOpt data file `4x4x4_48_201`.

[17] G. Sidebottom, *logic*, source repository, https://github.com/gsidebottom/logic (2026).